

Step 3 — Test-Coverage Reviewer + Supervisor — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# agents/test_coverage_reviewer.py
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Finding(BaseModel):
    severity: str
    message: str

class CoverageReview(BaseModel):
    findings: list

def run_coverage_review(diff):
    changed_files = [line for line in diff.splitlines() if line.startswith("diff")]
    has_test_file = any("test_" in f for f in changed_files)
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(CoverageReview)
    return model.invoke(f"Changed files: {changed_files}. Has a test file changed? {has_test_file}. Please review this diff for test coverage:\n{diff}")

# supervisor.py
def supervisor(diff):
    security = run_security_review(diff)
    style = run_style_review(diff)
    coverage = run_coverage_review(diff)
    all_findings = security.findings + style.findings + coverage.findings
    return "\n".join(f"[{f.severity}] {f.message}" for f in all_findings)
```

This works and proves all 3 reviewers plus a compiling step. It always runs every reviewer on every diff (no routing), and "compiling" is just concatenation with no dedup or sorting — both real gaps a code-review interviewer would ask about immediately.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — file-based routing, typed models, sorted/deduped compiling

```
# agents/test_coverage_reviewer.py
import re
from pydantic import BaseModel, Field
from langchain_openai import ChatOpenAI

class Finding(BaseModel):
    severity: str = Field(description="One of: critical, warning, info")
    message: str

class CoverageReview(BaseModel):
    findings: list[Finding]

def extract_changed_files(diff: str) -> list[str]:
    return re.findall(r'diff --git a/(\\S+) b/\\S+', diff)

def run_coverage_review(diff: str) -> CoverageReview:
    changed_files = extract_changed_files(diff)
    source_files = [f for f in changed_files if not ("test_" in f or f.startswith("tests_"))]
    test_files = [f for f in changed_files if "test_" in f or f.startswith("tests_")]

    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(CoverageReview)
    return model.invoke(
        f"Source files changed: {source_files}\nTest files changed: {test_files}\n"
        f"If source files changed with no corresponding test file changes, flag it as a\n"
        f"warning. Diff:\n{diff}"
    )
```

```
# routing.py
def route_reviewers(diff: str) -> list[str]:
    changed_files = extract_changed_files(diff)
    reviewers = []
    if any(not f.endswith(("md", ".txt")) for f in changed_files):
        reviewers.append("security")
    if any(f.endswith(("py", ".js", ".ts")) for f in changed_files):
        reviewers.append("style")
    if any(f.endswith(("py", ".js", ".ts")) for f in changed_files):
```

```

    reviewers.append("test_coverage")
return reviewers

```

```

# supervisor.py
SEVERITY_ORDER = {"critical": 0, "warning": 1, "info": 2}

def compile_review(all_findings: list[Finding]) -> str:
    seen = set()
    unique = []
    for finding in all_findings:
        key = finding.message.strip().lower()
        if key not in seen:
            seen.add(key)
            unique.append(finding)

    unique.sort(key=lambda f: SEVERITY_ORDER.get(f.severity, 99))
    lines = [f"[{f.severity.upper()}] {f.message}" for f in unique]
    return "\n".join(lines) if lines else "No issues found."

def run_codeguard(diff: str) -> str:
    applicable = route_reviewers(diff)
    all_findings: list[Finding] = []
    if "security" in applicable:
        all_findings += run_security_review(diff).findings
    if "style" in applicable:
        all_findings += run_style_review(diff).findings
    if "test_coverage" in applicable:
        all_findings += run_coverage_review(diff).findings
    return compile_review(all_findings)

```

Difference from Basic: `route_reviewers` skips Security on a docs-only diff and skips Style/Test-Coverage on non-code files, directly matching the problems-table complaint about wasted reviewer runs. `compile_review` sorts by severity and removes exact-duplicate messages, instead of just concatenating everything in arrival order. Still a real gap: exact-string dedup only catches *identical* messages — two reviewers describing the same underlying problem in different words (a very real case in practice) still show up as 2 separate findings.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — an LLM compiling pass instead of a mechanical merge

```
from pydantic import BaseModel

class CompiledFinding(BaseModel):
    severity: str
    message: str

class CompiledReview(BaseModel):
    findings: list[CompiledFinding]

def compile_review(all_findings: list[Finding]) -> CompiledReview:
    if not all_findings:
        return CompiledReview(findings=[])

    raw = "\n".join(f"- [{f.severity}] {f.message}" for f in all_findings)
    model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output
    return model.invoke(
        "You are the lead reviewer compiling 3 specialist reviewers' findings into a clear review. Merge any findings that describe the same underlying issue (even if worded differently) into one, keep the most severe rating for multiple items, and sort by severity (critical first). Drop anything trivial or redundant.\n\nRaw findings:\n" + raw
    )
```

This directly closes the gap plain string-matching left: two reviewers phrasing the same problem differently now genuinely merge, because a model — not exact string equality — is doing the comparison. It costs one extra API call per review, which is a reasonable trade for what the problems table explicitly asks for: "not a copy-paste of three separate reports."

Approach 2 — the optional bounded double-check loop

```
MAX_CLARIFICATION_ROUNDS = 1

class SupervisorState(TypedDict):
    diff: str
    findings: list[Finding]
```

```

clarification_rounds: int

def needs_clarification(finding: Finding) -> bool:
    # a simple, explicit heuristic: very short reasoning is a sign the
    # reviewer didn't actually explain itself
    return len(finding.message) < 20

def supervisor_node(state: SupervisorState):
    unclear = [f for f in state["findings"] if needs_clarification(f)]
    if unclear and state["clarification_rounds"] < MAX_CLARIFICATION_ROUNDS:
        return Command(goto="clarify", update={"clarification_rounds": state["clarification_rounds"] + 1})
    return Command(goto="compile")

```

This reuses Project 4 Step 2's exact discipline: a hard round limit (`MAX_CLARIFICATION_ROUNDS`), and a state field (`clarification_rounds`) that actually gets checked before allowing another round — without it, an unclear finding that never gets clarified successfully would otherwise loop the Supervisor forever, the same failure mode ContentForge's Writer ↔ Reviewer loop had to solve first.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's compiling step only catches literally identical messages, and there's no way for the Supervisor to push back on a vague finding — every reviewer's first answer is final. Approach 1 fixes compiling specifically: near-duplicate findings genuinely merge, because judgment, not string equality, drives the merge. Approach 2 is a separate, optional capability — giving the Supervisor a bounded way to ask a reviewer to clarify before finalizing, reusing a pattern (hard round limit) this curriculum already proved necessary once.

Which one should you actually write? Approach 1's LLM-based compiling — genuine near-duplicate findings are common enough in practice (2 reviewers both noticing "this function has no error handling," worded differently) that mechanical string matching under-merges in a way that's actually noticeable in the final review. Approach 2's clarification loop is explicitly optional in the README ("if you want to go further") — worth building once Approach 1 is solid, as the clearest demonstration that Project 4's bounded-loop lesson generalized to a second, unrelated system, which is this entire project's point.